

# Retiring niebloids

Document #: P3136R1  
Date: 2024-11-18  
Project: Programming Language C++  
Audience: LWG  
Reply-to: Tim Song  
<[t.canens.cpp@gmail.com](mailto:t.canens.cpp@gmail.com)>

## § 1 Abstract

---

This paper proposes that we respecify the algorithms in `std::ranges` that are currently so-called niebloids to be actual function objects.

## § 2 Revision history

---

— R1: Incorporated LWG review feedback.

## § 3 Background

---

“Niebloid” is the informal name given to the “function templates” in `std::ranges` with the special property described in 26.2 [\[algorithms.requirements\]](#) p2:

- 2 The entities defined in the `std::ranges` namespace in this Clause are not found by argument-dependent name lookup (6.5.4 [\[basic.lookup.argdep\]](#)). When found by unqualified (6.5.3 [\[basic.lookup.unqual\]](#)) name lookup for the *postfix-expression* in a function call (7.6.1.3 [\[expr.call\]](#)), they inhibit argument-dependent name lookup.

[ *Example 1*:

```
void foo() {  
    using namespace std::ranges;  
    std::vector<int> vec{1,2,3};  
    find(begin(vec), end(vec), 2);           // #1  
}
```

The function call expression at #1 invokes `std::ranges::find`, not `std::find`, despite that (a) the iterator type returned from `begin(vec)` and `end(vec)` may be associated with namespace `std` and (b) `std::find` is more specialized (13.7.7.3 [\[temp.func.order\]](#)) than `std::ranges::find` since the former requires its first two parameters to have the same type.

— *end example* ]

This example also shows the reason for this requirement: because ranges algorithms accept iterator-sentinel pairs, they are almost always less specialized than their classic counterparts and therefore would

otherwise lose overload resolution.

Of course, there's nothing in the core language that actually allow a *function template* to have this property. Instead, all major implementations implement them as function objects, aided by the ban on explicit template argument lists in 26.2 [\[algorithms.requirements\]](#) p15:

- <sup>15</sup> The well-formedness and behavior of a call to an algorithm with an explicitly-specified template argument list is unspecified, except where explicitly stated otherwise.

The original idea behind this specification was that there might eventually be a core language change and/or some compiler extension that would allow implementing them as function templates.

## § 4 Discussion

---

We should bite the bullet and just specify niebloids as function objects.

### § 4.1 No language extension has materialized

Four years after C++20, the language extension has not materialized, and is unlikely to do so. It is hard to motivate a language change when the function-object based implementation works just as well.

### § 4.2 Implementations have converged

While there were originally implementation divergence on whether niebloids should be CPO-like semiregular function objects or noncopyable ones to more closely emulate function templates, the major implementations [have now converged](#) on semiregularity. We should standardize this existing practice.

### § 4.3 The status quo discourages reasonable code

Consider:

```
auto x = std::views::transform(std::ranges::size);  
auto y = std::views::transform(std::ranges::distance);  
auto z = std::views::transform(std::ref(std::ranges::distance));  
auto w = std::views::transform([](auto&& r) { return std::ranges::distance(r); });
```

x is valid because size is a CPO. y might not be, because distance is a niebloid, and until last October, at least one major implementation disallowed copying niebloids. z is de-facto valid - as long as std::ranges::distance is an object, std::ref should work on it, and then reference\_wrapper's call operator kicks in. w is valid but excessively verbose.

There's nothing inherently objectionable about y; we are not doing our users a service by disallowing it on paper. There's no reason to insist on writing a lambda.

### § 4.4 Unresolved wording issues

The difference between a function object and a set of function templates goes beyond the suppression of ADL and the inability to specify a template argument list. For example, function objects do not overload. The current wording is therefore technically insufficient to permit the function-object implementation. Instead of trying to further weasel-word this, I think we should just admit that niebloids are function objects.

## § 5 Wording

---

This wording is relative to [\[N4971\]](#).

1. Strike 16.3.3.3.5 [\[customization.point.object\]](#) p6:

[ Drafting note: This sentence isn't really true - what concept does `views::single` constrain its return type with? In any event, it doesn't have any normative effect on its own; if there is a constraint then the CPO's specification will specify it. ]

- 6 ~~Each customization point object type constrains its return type to model a particular concept.~~

2. Add the following subclause to 16.3.3 [\[conventions\]](#):

### § 16.3.3.? Algorithm function objects [nieblroid]

- 1 An *algorithm function object* is a customization point object (16.3.3.3.5 [\[customization.point.object\]](#)) that is specified as one or more overloaded function templates. The name of these function templates designates the corresponding algorithm function object.
- 2 For an algorithm function object `o`, let `S` be the corresponding set of function templates. Then for any sequence of arguments `args...`, `o(args...)` is expression-equivalent to `s(args...)`, where the result of name lookup for `s` is the overload set `S`.

[ *Note 1:* Algorithm function objects are not found by argument-dependent name lookup (6.5.4 [\[basic.lookup.argdep\]](#)). When found by unqualified (6.5.3 [\[basic.lookup.unqual\]](#)) name lookup for the *postfix-expression* in a function call (7.6.1.3 [\[expr.call\]](#)), they inhibit argument-dependent name lookup.

[ *Example 1:*

```
void foo() {
    using namespace std::ranges;
    std::vector<int> vec{1,2,3};
    find(begin(vec), end(vec), 2);           // #1
}
```

The function call expression at #1 invokes `std::ranges::find`, not `std::find`. — *end example* ]

— *end note* ]

3. Edit 24.4.4.1 [\[range.iter.ops.general\]](#) as indicated:

- 2 ~~The function templates defined in 24.4.4 [\[range.iter.ops\]](#) are not found by argument-dependent name lookup (6.5.4 [\[basic.lookup.argdep\]](#)). When found by unqualified (6.5.3 [\[basic.lookup.unqual\]](#)) name lookup for the *postfix-expression* in a function call (7.6.1.3 [\[expr.call\]](#)), they inhibit argument-dependent name lookup.~~

~~[Example 2:~~

```
void foo() {  
    using namespace std::ranges;  
    std::vector<int> vec{1,2,3};  
    distance(begin(vec), end(vec)); // #1  
}
```

~~The function call expression at #1 invokes `std::ranges::distance`, not `std::distance`, despite that (a) the iterator type returned from `begin(vec)` and `end(vec)` may be associated with namespace `std` and (b) `std::distance` is more specialized (13.7.7.3 [\[temp.func.order\]](#)) than `std::ranges::distance` since the former requires its first two parameters to have the same type. —end example]~~

- 3 ~~The number and order of deducible template parameters for the function templates defined in 24.4.4 [\[range.iter.ops\]](#) is unspecified, except where explicitly stated otherwise.~~
- 2 The entities defined in 24.4.4 [\[range.iter.ops\]](#) are algorithm function objects (16.3.3.? [\[niebloid\]](#)).

4. Edit 26.2 [\[algorithms.requirements\]](#) p2 as indicated:

- 2 ~~The entities defined in the `std::ranges` namespace in this Clause are not found by argument-dependent name lookup (6.5.4 [\[basic.lookup.argdep\]](#)). When found by unqualified (6.5.3 [\[basic.lookup.unqual\]](#)) name lookup for the *postfix-expression* in a function call (7.6.1.3 [\[expr.call\]](#)), they inhibit argument-dependent name lookup.~~

~~[Example 1:~~

```
void foo() {  
    using namespace std::ranges;  
    std::vector<int> vec{1,2,3};  
    find(begin(vec), end(vec), 2); // #1  
}
```

~~The function call expression at #1 invokes `std::ranges::find`, not `std::find`, despite that (a) the iterator type returned from `begin(vec)` and `end(vec)` may be associated with namespace `std` and (b) `std::find` is more specialized (13.7.7.3 [\[temp.func.order\]](#)) than `std::ranges::find` since the former requires its first two parameters to have the same type. —end example]~~

- 2 The entities defined in the `std::ranges` namespace in this Clause and specified as function templates are algorithm function objects (16.3.3.? [\[niebloid\]](#)).

## § 6 References

---

[N4971] Thomas Köppe. 2023-12-18. Working Draft, Programming Languages — C++. <https://wg21.link/n4971>